

- *AeonWave 4D* is a low-cost audio library for Windows and Linux by Adalin B.V.
- *FMOD Studio* is an audio authoring tool that features a “pro audio” look and feel (<http://www.fmod.org>). A full-featured runtime 3D audio API allows assets created in FMOD Studio to be rendered in real time on the Windows, Mac, iOS and Android platforms.
- *Miles Sound System* is a popular audio middleware solution by Rad Game Tools (<http://www.radgametools.com/miles.htm>). It provides a powerful audio processing graph and is available on virtually every gaming platform imaginable.
- *Wwise* is a 3D audio rendering engine by Audiokinetic (<https://www.audiokinetic.com>). It is notably not based around the concepts and features of a multi-channel mixing console, but rather presents the sound designer and programmer with a unique interface based on game objects and events.
- The *Unreal Engine* of course provides its own 3D audio engine and powerful integrated tool chain (<http://www.unrealengine.com>). For an in-depth look at Unreal’s audio feature set and tools, see [45].

14.6 Game-Specific Audio Features

On top of the 3D audio rendering pipeline, games typically implement all sorts of game-specific features and systems. Some examples include:

- *Split-screen support*. Multiplayer games that support split-screen play must provide some mechanism that allows *multiple listeners* in the 3D game world to *share* a single set of speakers in the living room.
- *Physics-driven audio*. Games that support dynamic, physically simulated objects like debris, destructible objects and rag dolls require a means of playing appropriate audio in response to impacts, sliding, rolling and breaking.
- *Dynamic music system*. Many story-driven games require the music to adapt in real time to the mood and tension of events in the game.
- *Character dialog system*. AI-driven characters seem a great deal more realistic when they speak to one another and to the player’s character.
- *Sound synthesis*. Some engines continue to provide the ability to synthesize sounds “from scratch” by combining various kinds of waveforms (sinusoid, square, sawtooth, etc.) at various volumes and frequencies. Advanced synthesis techniques are also becoming practical for use in real-time games. For example:

- *Musical instrument synthesizers* reproduce the natural sound of an analog musical instrument without the use of pre-recorded audio.
 - *Physically based sound synthesis* encompasses a broad range of techniques that attempt to accurately reproduce the sound that would be made by an object as it physically interacts with a virtual environment. Such systems make use of the contact, momentum, force, torque and deformation information available from a modern physics simulation engine, in concert with the properties of the material from which the object is made and its geometric shape, in order to synthesize suitable sounds for impacts, sliding, rolling, bending and so on. Here are just a few links to research on this fascinating topic: <http://gamma.cs.unc.edu/research/sound>, http://gamma.cs.unc.edu/AUDIO_MATERIAL, <http://www.cs.cornell.edu/projects/sound>, and <https://ccrma.stanford.edu/~bilbao/booktop/node14.html>.
 - *Vehicle engine synthesizers* aim to reproduce the sounds made by a vehicle, given inputs such as the acceleration, RPM and load placed on a virtual engine, and the mechanical movements of the vehicle. (The vehicle chase sequences in Naughty Dog's three *Uncharted* games all used various forms of dynamic engine modeling, although technically these systems were not synthesizers, because they produced their output by cross-fading between various pre-recorded sounds.)
 - *Articulatory speech synthesizers* produce human speech "from scratch" via a 3D model of the human vocal tract. VocalTractLab (<http://www.vocaltractlab.de>) is a free tool that allows students to learn about and experiment with vocal synthesis.
- *Crowd Modeling*. Games that feature crowds of people (audiences, city dwellers, etc.) require some means of rendering the sound of that crowd. This is not as simple as playing lots and lots of human voices over top of one another. Instead, it is usually necessary to model the crowd as multiple layers of sounds, including a background ambience plus individual vocalizations.

We can't possibly cover everything from the above list in one chapter. But let's spend a few more pages covering some of the most common game-specific features.

14.6.1 Supporting Split-Screen

Supporting split-screen multiplayer is a tricky problem, because you have *multiple listeners* in the virtual game world, but they must *share* a single set of speakers in the player's living room. If you simply pan the sounds multiple times, once for each listener, and then mix the results into the speakers evenly, the result won't always sound sensible. There is no perfect solution: For example, if player A is standing right next to an explosion and player B is standing far away, the person playing player B will still hear the explosion loud and clear. The best a game can do is cobble together a hybrid solution, in which some sounds are handled in a "physically correct" way and others are "fudged" in order to produce the most sensible-sounding experience for the players.

14.6.2 Character Dialog

Even if we've created characters for our game that look like photographs of real human beings, and even if they move in astoundingly realistic ways, they still won't seem real to the player until they can *speak* realistically. Speech communicates information crucial to gameplay. It's a central story-telling tool. And it cements the emotional bond between the human player and the characters in the game. Speech can also be the deciding factor in the player's perception of *intelligence* among the AI-controlled characters in a game.

At the Game Developer's Conference (GDC) in 2002, Chris Butcher and Jaime Griesemer of Bungie gave a talk entitled, "The Illusion of Intelligence: The Integration of AI and Level Design in Halo" (<http://bit.ly/1g7FbhD>). In their talk, they shared an anecdote about how important speech can be to communicating the motivations of an AI-driven character to the player. In *Halo*, when the Elite leader of a squad of Covenant was killed, the grunts would all run away in fear. In playtest after playtest, no one seemed to understand that it was the killing of the Elite that had triggered the grunts to flee. Finally, the grunts were given lines of dialog saying something to the effect of, "Leader dead—run away!" Only then did play testers start to really grok what was going on!

In this section, we'll explore some of the fundamental subsystems you'll find in the character dialog system of pretty much any character-based game. We'll also discuss some of the specific techniques and technologies used by Naughty Dog to create rich, realistic conversations in *The Last of Us*. For more information and in-game videos of Naughty Dog's character dialog system in action, check out the talk I gave at GDC 2014 entitled, "Context-Aware Character Dialog in *The Last of Us*," available in PDF and QuickTime formats at

<http://www.gameenginebook.com>.

14.6.2.1 Giving a Character a Voice

It's easy enough to give a game character a voice—simply play an appropriate pre-recorded sound whenever the character needs to speak. However, things are never quite that simple. The dialog system in a game engine is typically a reasonably complex beast. Here are just a few of the reasons why:

- We need a way to catalog all the possible lines of dialog that each character might be called upon to say, and give each of these lines some kind of unique id so that the game can trigger them when needed.
- We need to make sure that each uniquely identifiable character in the game speaks with a recognizable and consistent voice. For example, each of the hunters in the Pittsburgh section of *The Last of Us* was assigned to one of eight unique voices, so that no two hunters in a battle would sound the same.
- We may not know a priori which character is going to be called upon to say a specific line, so we often need to record the same line multiple times, spoken by various voice actors, so that the appropriate voice can be used to say the line when needed.
- We also usually want a lot of variety in the things that are said. So most dialog systems provide a means of selecting specific lines at random from a pool of possibilities.
- Speech audio assets tend to be of relatively long duration, which means they occupy a lot of memory. Many lines of dialog are part of cinematic sequences, and hence are only spoken once in the entire game. For these reasons, it's usually wasteful to store speech assets in memory. Instead, it's typical for speech audio assets to be *streamed* on demand (see Section 14.5.7.4).

Usually other vocal sounds, like the “efforts” made when lifting something heavy, jumping over an obstacle or getting punched in the gut, are handled by the same system that handles spoken dialog. This is done largely because a character's efforts need to match his or her spoken voice. So we may as well leverage the dialog system to produce effort sounds as well.

14.6.2.2 Defining a Line of Dialog

Most dialog systems introduce a level of indirection between a *request* to speak and the choice of the particular *audio clip* to play. The game programmer or designer requests *logical* lines of dialog, each of which is represented by a unique

identifier such as a string, or better yet a hashed string id (see Section 6.4.3.1). The sound designers can then “fill out” each logical line with one or more audio clips in order to provide the necessary variety both in voice quality and in terms of what exactly is said.

For example, let’s imagine a logical line in which the character says something to the effect of, “I’m out of ammo.” We’ll assign this logical dialog line the unique id `'line-out-of-ammo`, where the leading single quote indicates a hashed string id. Let’s assume also that there are ten different characters that might say this line: the player character (call him “drake”), the player’s sidekick (call her “elena”) and up to eight enemy characters (call them “pirate-a” through “pirate-h”). We’ll need some kind of data structure to define all the physical audio assets that make up this one logical line of dialog.

At Naughty Dog, sound designers use the Scheme programming language to define logical dialog lines using a custom syntax. We’ll use a similar syntax in our example below. However, the specifics of the implementation are not important here. All we’re interested in is the structure of the data itself:

```
(define-dialog-line 'line-out-of-ammo
  (character 'drake
    (lines
      drk-out-of-ammo-01    ;; "Dammit, I'm out!"
      drk-out-of-ammo-02    ;; "Crap, need more bullets."
      drk-out-of-ammo-03    ;; "Oh, now I'm REALLY mad."
    )
  )
  (character 'elena
    (lines
      eln-out-of-ammo-01    ;; "Help, I'm out!"
      eln-out-of-ammo-02    ;; "Got any more bullets?"
    )
  )
  (character 'pirate-a
    (lines
      pira-out-of-ammo-01    ;; "I'm out!"
      pira-out-of-ammo-02    ;; "Need more ammo!"
      ;; ...
    )
  )
  ;; ...
  (character 'pirate-h
    (lines
      pirh-out-of-ammo-01    ;; "I'm out!"
      pirh-out-of-ammo-02    ;; "Need more ammo!"
```

```

        )
    )
)

```

Rather than define your dialog lines in one monolithic data structure like the one shown above, it's usually better to break the lines out into separate files by character. For example, all of Drake's lines can be managed in one file, Elena's in another file and all the pirates' lines can be stored in a third file. This helps to prevent the sound designers from stepping on each others' toes. It also means that we can manage our memory more efficiently. For example, if there are no pirates in a given section of the game, there's no need to keep the data for the pirates' dialog lines in memory. It's also a good idea to split the dialog data up by level, for the same reason.

14.6.2.3 Playing a Line of Dialog

Given this data, the dialog system can easily convert a request for a logical line of dialog such as 'line-out-of-ammo' into a specific audio clip. It simply looks up the character's specific voice id in the table, and then makes a random choice amongst the various possible lines for that character.

It's usually a good idea to put in place some kind of mechanism to ensure that lines aren't repeated too often. One way to accomplish this is to store the indices of the various lines in an array and then randomly shuffle its contents. To select a line, we simply cycle through the shuffled array in order. Once all possible lines have been exhausted, we reshuffle the array, taking care that the most recently played line doesn't end up in the first slot. This prevents all repetition while keeping the line selections sounding random.

Dialog line requests are typically made by gameplay code in C++, Java, C# or whatever language your game is written in. Game designers may also request lines of dialog via script (Lua, Python, etc.). The dialog system's API is usually designed with simplicity of use in mind. If an AI programmer or game designer has to jump through a lot of hoops just to get a line of dialog to play, you may discover that your characters are uncannily silent! It's best to provide a simple, fire-and-forget interface. Leave all the hard work to the programmer who is crafting the dialog system.

For example, in *Uncharted 3: Drake's Deception*, C++ code could request a character to play a line of dialog by calling a simple `PlayDialog()` member function of the `Npc` class. These calls would be peppered throughout the AI decision-making code in order to trigger appropriate lines of dialog at key moments in the game. For example:

```

void Skill::OnEvent(const Event& evt)
{
    NPC* pNPC = GetSelf(); // grab a pointer to the NPC

    switch (evt.GetMessage())
    {
    case SID("player-seen"):
        // play a line of dialog...
        pNPC->PlayDialog(SID("line-player-seen"));
        // ... and move to closest cover
        pNPC->MoveTo(GetClosestCover());
        break;
    // ...
    }

    // ...
}

```

14.6.2.4 Priority and Interruption

What happens if a character is asked to speak while he's already speaking? What if he receives more than one speech command on the same frame? In both cases, a *priority system* is a good way to resolve ambiguities.

To implement such a system, we simply assign each line of dialog to a priority level. When a request to say a line of dialog comes in, the system looks at the priority of the currently playing line if any, and the priorities of the line or lines that have been requested this frame. It finds the highest priority line among these. If the currently playing line “wins,” it continues to play and the requested lines are ignored. If one of the requested lines is higher priority than the current line, or if the character isn't speaking yet, the new line plays, interrupting the current line if necessary.

Implementing the interruption of the speech itself is actually a bit tricky. We can't perform a cross-fade (i.e., fade the volume of the playing sound down and the new sound up) because this sounds strange and wrong when applied to the speech of a single character. Ideally, we'd want to play at least some kind of glottal stop sound just prior to starting the new line. It might even be appropriate to play a short phrase indicating that the character is surprised and/or annoyed by the interruption, and *then* play the new line of dialog. The dialog system in *The Last of Us* doesn't do any of these fancy things. It simply stops the current line and immediately plays the new one. This sounds pretty good most of the time. Of course, each game has its own unique speech patterns, and what works in one game may not work well in another. So as the saying goes, “Your mileage may vary.”

14.6.2.5 Conversations

In *The Last of Us*, Naughty Dog wanted the enemy NPCs to sound like they're having real conversations with one another. This meant that the characters would need to be capable of saying relatively long chains of lines, with back-and-forth banter between two or more characters. Likewise, in *Uncharted 4: A Thief's End* and *Uncharted: The Lost Legacy*, we wanted the characters to have conversations while driving around Madagascar and India in their jeep. These conversations could even be interrupted (for example by the player deciding to exit the jeep to explore the area), and would continue where they left off when the player returned to the vehicle.

Conversations in *The Last of Us*, *Uncharted 4* and *The Lost Legacy* are constructed from logical *segments*. Each segment corresponds to one logical line, spoken by one particular actor in the conversation. Each segment is given a unique id, and the segments are *chained* together into a conversation via these ids. As an example, let's see how we would define the following conversation:

A: "Hey, did you find anything?"

B: "No, I've been looking for an hour and I ain't found nothin'."

A: "Well then shut up and keep looking!"

This conversation could be expressed in the Naughty Dog conversation system as follows:

```
(define-conversation-segment 'conv-searching-for-stuff-01
  :rule []
  :line 'line-did-you-find-something
        ;; "Hey, did you find anything?"
  :next-seg 'conv-searching-for-stuff-02
)
(define-conversation-segment 'conv-searching-for-stuff-02
  :rule []
  :line 'line-nope-not-yet
        ;; "I've been looking for an hour..."
  :next-seg 'conv-searching-for-stuff-03
)
(define-conversation-segment 'conv-searching-for-stuff-03
  :rule []
  :line 'line-shut-up-keep-looking
        ;; "Well then shut up and keep looking!"
)
```

This syntax might seem a bit verbose at first glance. But as we'll see in Section 14.6.2.8, breaking the conversation out like this gives us a great deal of

flexibility. For example, it allows branching conversations to be defined in a natural and reasonably convenient way.

14.6.2.6 Interrupting Conversations

We saw in Section 14.6.2.4 that a simple priority system can be used to handle interruptions and to resolve contention when more than one logical line is requested simultaneously.

When *conversations* are in play, a priority system can still be used. But its implementation is a bit more complex in this case. For example, imagine a conversation between characters A and B. A says his line, then B says her line while A waits his turn. During the time that B is speaking, A is asked to play an entirely different line of dialog. He's not technically speaking, so by the rules for dialog prioritization, applied to each character individually, there would be no problem and the line would play. But this could sound very jarring, depending on what's being said.

A: "Hey, did you find anything?"

B: "No, I've been looking for an hour and..."

A: "Look, a shiny object!"
(*interruption by an unrelated line of dialog*)

B: "...I ain't found nothin'."

To overcome this issue on *The Last of Us*, we introduced the concept of conversations as "first-class entities." When a conversation is started, the system "knows" that each of the characters is involved in that conversation, even when he or she isn't speaking. Each conversation has a priority, and the prioritization rules are applied to entire conversations, not the individual lines on a per-character basis. So for example, when character A is asked to say, "Look, a shiny object!" the system knows that he is currently involved in the "Hey, did you find anything?" conversation. Presumably the line "Look, a shiny object!" is at the same or lower priority as the current conversation, so the interruption isn't allowed.

If the interrupting line is something higher priority like, "Holy cow, he's pointing a gun at us!" then the line *is* allowed to interrupt the existing conversation. In that case, all of the characters in the conversation are interrupted. The result is an interruption that sounds natural and intelligent.

A: "Hey, did you find anything?"

B: "No, I've been looking for an hour and..."

- A:** “Holy cow, he’s pointing a gun at us!”
(*interruption by a higher-priority conversation*)
- B:** “Get him!”
(*The original conversation is interrupted by the new one, and A and B go into combat mode.*)

14.6.2.7 Exclusivity

On *The Last of Us*, we also introduced the concept of *exclusivity*. Any line of dialog or conversation can be marked as either *non-exclusive*, *faction-exclusive* or *globally exclusive*. This mark-up controls how interruptions work for the given line or conversation.

- A *non-exclusive* line or conversation is permitted to play *over top* of other lines or conversations. For example, during a search for the player, it’s not a big problem if one hunter is mumbling to himself, “Huh, there’s nothing over here.” while another hunter is saying, “I’m getting tired of this.” The two hunters aren’t speaking to each other, so the overlap sounds perfectly natural.
- A *faction-exclusive* line or conversation interrupts all other lines or conversations within that character’s faction. For example, if the player (Joel) is spotted during a search, the hunter that saw him might say, “He’s over here!” The other hunters should immediately stop speaking, because we want to make it seem as if the hunters can hear one another, and also to communicate to the player that their collective focus has shifted. However, if Joel’s companion Ellie is whispering a warning to him at the time, we probably do not want to interrupt her. She is not part of the hunter gang, and what she has to say to Joel is relevant whether or not the hunters have spotted him.
- A *globally exclusive* line or conversation interrupts all other lines, across faction boundaries. This is useful in situations in which *every* character within earshot should react to hearing whatever is being said.

14.6.2.8 Choices and Branching Conversations

It’s often desirable to allow conversations to play out in different ways depending on what the player does, on what decisions the AI characters make and/or on other aspects of game world state. When authoring or editing conversations, the writers and sound designers would like to have control not only over which lines are said, but also over the logical conditions that control which branch of the conversation will be taken at any given moment during

gameplay. This puts the creative power in the hands of the people who need it, instead of forcing them to work through a programmer.

Naughty Dog implemented such a system for use on *The Last of Us*. It was inspired in part by an earlier system developed by Valve and described by Elan Ruskin in his talk, “Rule Databased for Contextual Dialog and Game Logic,” which he delivered at the Game Developer’s Conference in 2012. The talk is available here: <http://www.gdcvault.com/play/1015317/AI-driven-Dynamic-Dialog-through>. Naughty Dog’s conversation system differs from Valve’s in a number of significant ways, but the core idea behind both systems is similar. We’ll describe the Naughty Dog system here, since that’s the system with which the author has the most experience.

In Naughty Dog’s conversation system, each segment of a conversation can consist of one or more alternative lines of dialog. Each alternative within the segment carries with it a *selection rule*. If the rule evaluates to true, that alternative is selected; if the rule evaluates to false, the alternative is ignored.

A rule is comprised of one or more criteria. Each criterion is a simple logical expression that evaluates to a Boolean. The expressions (`'health > 5`) and (`'player-death-count == 1`) are examples of criteria. If more than one criterion is provided within a rule, they are logically combined using the Boolean AND operator. A rule only evaluates to true when *all* of its criteria evaluate to true.

Here’s an example of one segment of a conversation, with three alternatives that depend upon the health of the character doing the talking:

```
(define-conversation-segment 'conv-player-hit-by-bullet
  (
    :rule [ ('health < 25) ]
    :line 'line-i-need-a-doctor
          ;; "I'm bleeding bad... need a doctor!"
  )
  (
    :rule [ ('health < 75) ]
    :line 'line-im-in-trouble
          ;; "Now I'm in real trouble."
  )
  (
    :rule [ ] ;; no criteria acts as an "else" case
    :line 'line-that-was-close
          ;; "Ah! Can't let that happen again!"
  )
)
```

Branching Dialog

By breaking a conversation into segments, each of which contains one or more alternative lines, we open up the possibility of crafting branching conversations. For example, let's consider a conversation in which Ellie (the player's companion in *The Last of Us*) asks Joel (the player character) if he's all right when he's been shot at. If the player wasn't actually hit by the bullet, the conversation goes like this:

Ellie: "Are you OK?"

Joel: "Yeah, I'm fine."

Ellie: "Geez. Keep your head down!"

If Joel has been hit, the conversation plays out differently:

Ellie: "Are you OK?"

Joel: "(panting) Not exactly."

Ellie: "You're bleeding!"

We can express this branching conversation using the conversation syntax described above:

```
(define-conversation-segment 'conv-shot-at--start
  (
    :rule [ ]
    :line 'line-are-you-ok ;; "Are you OK?"
    :next-seg 'conv-shot-at--health-check
    :next-speaker 'listener ;; *** see comments below
  )
)

(define-conversation-segment 'conv-shot-at--health-check
  (
    :rule [ (('speaker 'shot-recently) == false) ]
    :line 'line-yeah-im-fine ;; "Yeah, I'm fine."
    :next-seg 'conv-shot-at--not-hit
    :next-speaker 'listener ;; *** see comments below
  )
  (
    :rule [ (('speaker 'shot-recently) == true) ]
    :line 'line-not-exactly ;; "(panting) Not exactly."
    :next-seg 'conv-shot-at--hit
    :next-speaker 'listener ;; *** see comments below
  )
)
```



```

(define-conversation-segment 'conv-shot-at--not-hit
  (
    :rule [ ]
    :line 'line-keep-head-down ;; "Geez. Keep your head down!"
  )
)

(define-conversation-segment 'conv-shot-at--hit
  (
    :rule [ ]
    :line 'line-youre-bleeding ;; "You're bleeding!"
  )
)

```

Speaker and Listener

There's a subtle aspect to what's going on in the branching conversation above. At any given moment in a two-person conversation, one person is the speaker and the other is the listener. The roles of speaker and listener ping-pong back and forth as the conversation progresses. In the first segment of the conversation, 'conv-shot-at--start, Ellie is the speaker and Joel is the listener. When we chain to the next segment, 'conv-shot-at--health-check, we specify the value 'listener for the field :next-speaker. This tells the system to use the current listener (Joel) as the next segment's speaker, thereby reversing the roles. In that segment, we check whether the speaker has been shot recently via the criteria (('speaker 'shot-recently) == false) and (('speaker 'shot-recently) == true). But now Joel is the speaker, so everything works out as we'd expect.

An abstract speaker/listener system doesn't seem all that useful for a conversation between two principal characters like Joel and Ellie. But by keeping the definition of the conversation abstract, we gain a significant amount of flexibility. For one thing, we could use the same conversation specification to define a conversation in which Joel asks Ellie if she's OK. This works because the entire conversation is defined in a way that is independent of which character is saying each line. Moreover, for enemy characters it's absolutely essential that conversations be defined in a generic manner, because we don't know which specific characters will be doing the speaking a priori. For enemy battle chatter, we typically select a pair of characters dynamically and fire off the conversation. It has to work, no matter which two characters are selected.

The speaker/listener system can be extended to two- or three-person conversations. The Naughty Dog conversation system supported up to three listeners, although the vast majority of our conversations were between only

two characters.

Fact Dictionaries

The criteria within a rule reference symbolic quantities like 'health and 'player-death-count. These symbolic quantities are implemented under the hood as entries in a *dictionary* data structure—basically a table containing key-value pairs. We call these *fact dictionaries*. An example of a fact dictionary is shown in Table 14.1.

You may have noticed in Table 14.1 that each value in the dictionary has an associated data type. In other words, the values in the dictionary are *variants*. A variant is a data object that is capable of holding values of various types, much like a union in C or C++. However, unlike a union, a variant also stores information about the type of data it currently contains. This allows us to validate the type of a value prior to using it. It also lets us convert data from one type to another. For example, if our variant holds the integer value 42, we could ask the variant to return it to us as the floating-point value 42.0f instead.

In *The Last of Us*, each character has its own fact dictionary containing facts about the character itself like health, weapon type, awareness level and so on. Each “faction” of characters also has a fact dictionary. This allows us to express facts about the faction as a whole, like how many characters remain alive within the group. Finally, there is a singleton “global” fact dictionary that contains information about the game as a whole, without respect to faction. Things like the amount of time spent playing, the name of the current level or how many times the player has retried a particular task are all things that can go into the global fact dictionary.

Criterion Syntax

When writing a criterion, the syntax allows for facts to be pulled from any dictionary by name. For example, (('self 'health) > 5) tells the sys-

Key	Value	Data Type
'name	'ellie	StringId
'faction	'buddy	StringId
'health	82	int
'is-joels-friend	true	bool
...	...	...

Table 14.1. An example of a fact dictionary.

tem to grab the fact dictionary of the character itself, look up the value of the 'health fact in that dictionary and then check if it is greater than 5. Likewise, (('global 'seconds-playing) <= 23.5) instructs the system to look up the 'seconds-playing fact from the global fact dictionary, and check that it is less than or equal to 23.5 seconds.

If the user doesn't specify a dictionary explicitly, as in ('health > 5), the system searches for the named fact by following a predefined search order. Check the character's fact dictionary first. If that fails, try to find it in the dictionary that matches the character's faction. Finally, if all else fails, look for the fact in the global dictionary. This "search path" feature allows sound designers to be as brief as possible when writing criteria (albeit with the loss of some specificity and clarity in the rules).

14.6.2.9 Context-Sensitive Dialog

In *The Last of Us*, we wanted to have enemy characters call out the location of the player in an intelligent way. If the player is hiding in a store, the enemies should shout, "He's in the store!" If he's hiding behind a car, we want the bad guys to say, "He's behind that car!" This makes the characters sound incredibly intelligent, yet it turns out to be a relatively simple thing to implement.

To make this work, the sound designers mark up our game worlds with regions. Each region is tagged with one of two kinds of *location tags*. A *specific* tag marks the region with a very specific location like "behind the counter" or "by the cash register." A *general* tag marks the region with a more general location like "in the store" or "in the street."

To determine which line of dialog to play, the system determines within which region the player is located, and within which region the enemy NPC is located. If they are both in the same *general* region, the player's *specific* tag is used to select dialog lines. When the NPC and player reside in *different* general regions, we fall back to using the player's general region tag to select the line. So if the enemy and the player are both in the store, we might select a line like, "He's by the window!" But if the NPC is in the store and the player is out in the street, we might hear the NPC say, "He's out in the street! Get him!" See Figure 14.42 for an illustration of how this system works.

This very simple system proved incredibly powerful. It was difficult to set up due to the sheer number of combinations of lines that had to be recorded and configured, but the final result in-game was worth the effort.

14.6.2.10 Dialog Actions

Lines of dialog delivered without body language usually look uncanny and unrealistic. Some dialog lines are delivered as part of a full-body animation—an in-game cinematic for example. But some lines must be delivered while the

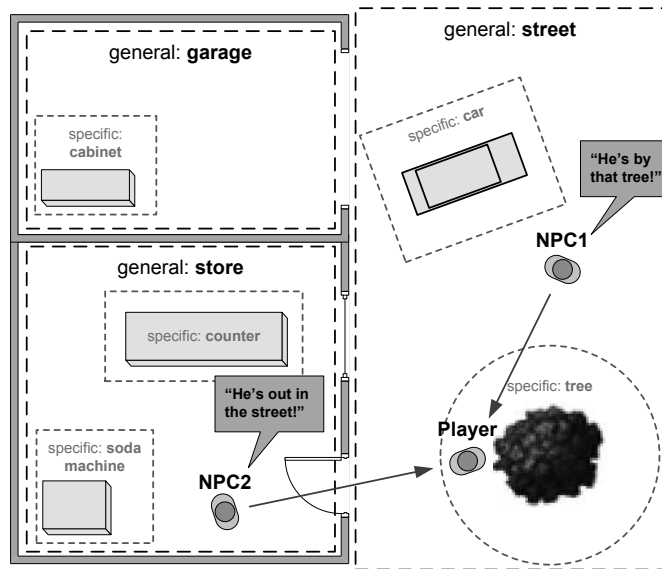


Figure 14.42. General and specific regions for context-sensitive dialog line selection.

character is busy doing something else, like walking, running or firing their weapon. Ideally we'd like to spice up such lines of dialog with some gestures to breathe life into them.

On *The Last of Us*, we implemented a gesture system using additive animation technology (see Section 12.6.5). These gestures could be explicitly called out by C++ code or script. In addition, each line of dialog could have a script associated with it whose timeline was synchronized with the audio. This allowed us to trigger gestures at precise moments during key lines of dialog.

14.6.3 Music

Music is an incredibly important aspect of pretty much any good game. It sets the tone, drives the player's sense of tension, and can make (or break) an emotional scene. A game engine's music system is typically charged with the following duties:

- Provide the ability to play back music tracks as streaming audio clips (because music clips are almost always too large to fit in memory).
- Provide musical variety.
- Match the music to the events occurring in the game.
- Seamlessly transition from one piece of music to the next.

- Mix the audio with the other sounds in the game in a suitable and pleasing manner.
- Allow music to be temporarily ducked to enhance the audibility of specific sounds or conversations in-game.
- Permit brief pieces of music or sound effects known as *stingers* to temporarily interrupt the currently playing music track.
- Allow music to be paused and restarted. (You don't need a full orchestra playing a grandiose theme during every single second of gameplay, you know!)

We generally expect the music to change to match the changing levels of tension and/or emotional moods of the events happening in the game. One way to accomplish this is to create multiple *playlists*, each of which is intended for a different level of tension or emotional mood in the game. Each playlist contains one or more pieces of music, from which selections may be made randomly or sequentially. As the tension and mood change in the game—as battles begin and end, touching cutscenes come and go and so on—the music system detects these changes and selects new music playlists as appropriate. Some games implement a “stack” of music selections at increasing tension levels—calm music for when no enemies are around, tense music when the player is approaching an unsuspecting group of enemies, startling music at first contact and fast-paced music during battle.

Stingers are another way to match the music to the events in the game. A stinger is a short musical clip or sound effect that can temporarily interrupt the currently playing music track, or play over top of it while the main track's volume is ducked down. For example, the first time the player makes line-of-sight with a new enemy, we might want to play an ominous “rumbling” sound to give the player a cue that danger is near. Or when the player dies, we may want to quickly switch to a snippet of “death music.” Both of these are situations in which a stinger might be used.

Transitioning smoothly between different music streams is somewhat of a challenge. We cannot blindly cross-blend between totally unrelated pieces of music and expect it to always sound good. The tempos of the two pieces may not match, and the “beat” of one piece of music might not line up with that of the next. The key is to time each transition properly. A rapid cross-fade can be useful if the tempos don't match; a longer cross-fade might work well if the tempos are nearly identical. This takes some trial and error to get right. Even getting a piece of music to loop properly requires some tweaking by the sound engineer.

The topic of game music is a broad one, and we can't really do it justice here. If you are interested in learning more, [45] is a great book to start with.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part IV

Gameplay



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Introduction to Gameplay Systems

Up until now, everything we've talked about in this book has focused on technology. We've learned that a game engine is a complex, layered software system built on top of the hardware, drivers and operating system of the target machine. We've seen how low-level engine systems provide services that are required by the rest of the engine; how human interface devices such as joypads, keyboards, mice and other devices can allow a human player to provide inputs to the engine; how the rendering engine produces 3D images on-screen; how the animation system allows characters and objects to move naturally; how the collision system detects and resolves interpenetrations between shapes; how the physics simulation causes objects to move in physically realistic ways; how the 3D audio engine renders a believable and immersive soundscape for our game world. But despite the wide range of powerful features provided by these components, if we were to put them all together, we *still* wouldn't have a game!

A game is defined not by its technology but by its *gameplay*. Gameplay can be defined as the overall experience of playing a game. The term *game mechanics* pins down this idea a bit more concretely—it is usually defined as the set of *rules* that govern the interactions between the various entities in the game. It also defines the *objectives* of the player(s), *criteria* for success and failure, the player character's *abilities*, the number and types of *non-player entities*